UNIBALSAS
Centro Universitário

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS
Unidade Curricular: Algoritmos e Programação Avançada
Professor: Alex Rogaleski Marques

Gerenciador de Persistência de Dados em Arquivos Estruturados (CSV e JSON)

Trabalho Efetivo Discente (TED)

Marco Antonio Klepac Neto-
26.1.13779
Thulio Augusto Sousa Barros -
26.1.16657

Balsas – MA 2026

1 Identificação

Instituição	UniBalsas
Curso	Análise e Desenvolvimento de Sistemas
Disciplina	Algoritmos e Programação Avançada
Atividade	TED03 – Gerenciador de Persistência de Dados em Arquivos Estruturados (CSV e JSON)
Integrantes	Marco Antônio Klepac Neto Thulio Augusto Sousa Barros

2 Introdução e objetivos

Aplicações reais precisam guardar informações além do tempo de execução do programa. Uma das formas mais simples de fazer isso é a persistência em arquivos de texto estruturado, entre os quais se destacam o CSV (Comma-Separated Values), muito usado em planilhas, e o JSON (JavaScript Object Notation), amplamente usado na troca de dados entre sistemas. Manipular esses arquivos exige cuidados com a codificação de caracteres, com o fechamento correto dos recursos e com falhas de entrada e saída.

Este trabalho apresenta um módulo desenvolvido em Python que gerencia dados persistidos em arquivos CSV e JSON, converte os dados entre os dois formatos e protege as informações por meio de tratamento de exceções e de backup automático antes de qualquer alteração.

O objetivo geral é consolidar os conhecimentos sobre manipulação de arquivos em Python. Como objetivos específicos, destacam-se:

a) ler arquivos CSV e convertê-los para JSON, e o caminho inverso, sem perda de registros ou de valores; b) utilizar o gerenciador de contexto with open(...) com codificação UTF-8 em todas as aberturas de arquivo; c) tratar exceções de E/S, apresentando mensagens claras e evitando o encerramento inesperado; d) criar uma cópia de segurança do arquivo original antes de qualquer escrita ou atualização.

3 Descrição da solução

3.1 Organização do código

A solução foi organizada em módulos, cada um com uma responsabilidade, dentro do pacote persistencia, além de um menu de linha de comando (main.py) e de um script de testes (testes.py):

Arquivo	Responsabilidade
leitor.py	Leitura de CSV e JSON e validação do formato lido.
escritor.py	Gravação de CSV e JSON e atualização de registros.
conversor.py	Conversão CSV para JSON e JSON para CSV.
backup.py	Cópia de segurança do arquivo antes de alterá-lo.
erros.py	Exceções próprias e conversão de erros em mensagens ao usuário.
main.py	Menu interativo no terminal.
testes.py	Testes automáticos de casos válidos e de erro.

3.2 Principais funcionalidades

O módulo oferece leitura de CSV, leitura de JSON, conversão entre os dois formatos, gravação, atualização de registros por campo-chave, backup automático e tratamento de erros. Os dados são mantidos em memória como uma lista de dicionários, em que cada dicionário representa um registro e as chaves correspondem aos nomes das colunas. Essa estrutura é o ponto em comum entre os dois formatos.

3.3 Manipulação e persistência dos dados

No CSV, a primeira linha define o cabeçalho e cada linha seguinte vira um registro. Como o CSV não guarda tipos, todos os valores são lidos como texto, o que evita qualquer perda de informação na conversão para JSON. Na conversão inversa, o cabeçalho do CSV é formado pela união das chaves de todos os registros, na ordem em que aparecem; campos ausentes ficam vazios, e listas ou objetos aninhados são gravados como texto JSON dentro da célula. O JSON é gravado com indentação e com ensure_ascii desativado, para que acentos permaneçam legíveis no arquivo.

4 Implementação

4.1 Uso do gerenciador de contexto with open(...)

Todas as aberturas de arquivo utilizam with open(...) com codificação explícita. Assim, o arquivo é fechado automaticamente ao final do bloco, mesmo quando uma exceção interrompe a execução. A leitura do CSV usa utf-8-sig para aceitar arquivos exportados pelo Excel, que podem iniciar com a marca BOM, e newline vazio, conforme recomenda a documentação do módulo csv.

Código 1 – Função ler_csv (leitor.py)

```
def ler_csv(caminho, delimitador=","):
    """Lê um CSV e retorna uma lista de dicionários (valores como texto).

    Usa utf-8-sig para aceitar arquivos exportados pelo Excel (com BOM).
    """
    with open(caminho, "r", encoding="utf-8-sig", newline="") as arquivo:
        leitor = csv.DictReader(arquivo, delimiter=delimitador)
        if leitor.fieldnames is None:
            raise DadosInvalidosError("o arquivo CSV está vazio.")
        registros = []
        for numero, linha in enumerate(leitor, start=2):
            if None in linha:
                raise DadosInvalidosError(
                    f"a linha {numero} tem mais campos que o cabeçalho.")
            registros.append(linha)
    return registros
```

4.2 Leitura, escrita e conversão entre CSV e JSON

A função ler_json valida que o conteúdo seja uma lista de objetos. As funções de escrita recebem a lista de registros e gravam no formato de destino. As rotinas de conversão apenas combinam leitura e escrita, o que mantém cada módulo simples.

Código 2 – Função ler_json (leitor.py)

```
def ler_json(caminho):
    """Lê um JSON e retorna uma lista de dicionários."""
    with open(caminho, "r", encoding="utf-8") as arquivo:
        dados = json.load(arquivo)
    if isinstance(dados, dict):
        dados = [dados]
    if not isinstance(dados, list) or not all(isinstance(r, dict) for r in dados):
        raise DadosInvalidosError(
            "esperada uma lista de objetos (registros) no JSON.")
    return dados
```

Código 3 – Função escrever_json (escritor.py)

```
def escrever_json(caminho, registros):
    _preparar_destino(caminho)
    with open(caminho, "w", encoding="utf-8") as arquivo:
        json.dump(registros, arquivo, ensure_ascii=False, indent=2)
    print(f"[OK] {len(registros)} registro(s) gravado(s) em '{caminho}'.")
```

Código 4 – Função escrever_csv (escritor.py)

```
def escrever_csv(caminho, registros):
    colunas = _colunas(registros)
    if not colunas:
        raise DadosInvalidosError("não há dados para gravar no CSV.")
    _preparar_destino(caminho)
    with open(caminho, "w", encoding="utf-8", newline="") as arquivo:
        escritor = csv.DictWriter(arquivo, fieldnames=colunas)
```

```
        escritor.writeheader()
        for registro in registros:
            escritor.writerow({c: _para_celula(registro.get(c, ""))
                               for c in colunas})
    print(f"[OK] {len(registros)} registro(s) gravado(s) em '{caminho}'.")
```

Código 5 – Função csv_para_json (conversor.py)

```
def csv_para_json(origem, destino):
    registros = ler_csv(origem)
    escrever_json(destino, registros)
    return len(registros)
```

Código 6 – Função json_para_csv (conversor.py)

```
def json_para_csv(origem, destino):
    registros = ler_json(origem)
    escrever_csv(destino, registros)
    return len(registros)
```

4.3 Tratamento de exceções

O tratamento de erros foi centralizado em erros.py. A função descrever_erro converte cada tipo de exceção em uma mensagem clara, e a função executar_seguro executa uma operação capturando as exceções previstas, exibindo a mensagem ao usuário e devolvendo o resultado da tentativa. Dessa forma, o programa não encerra de forma inesperada. São tratados FileNotFoundError, PermissionError, UnicodeDecodeError, JSON e CSV malformados, pasta no lugar de arquivo, dados fora do formato esperado e falha de backup. As exceções UnicodeDecodeError e JSONDecodeError são subclasses de ValueError, por isso são verificadas antes de erros mais genéricos.

Código 7 – Função descrever_erro (erros.py)

```
def descrever_erro(erro, caminho):
    """Converte uma exceção em mensagem clara para o usuário."""
    if isinstance(erro, FileNotFoundError):
        return f"Arquivo não encontrado: '{caminho}'. Verifique o nome e o caminho."
    if isinstance(erro, PermissionError):
        return f"Sem permissão para acessar ou modificar '{caminho}'."
    if isinstance(erro, IsADirectoryError):
        return f"'{caminho}' é uma pasta, não um arquivo."
    if isinstance(erro, UnicodeDecodeError):
        return (f"Erro de codificação ao ler '{caminho}': o arquivo não está em UTF-8. "
                "Salve-o novamente como UTF-8.")
    if isinstance(erro, json.JSONDecodeError):
        return (f"JSON inválido em '{caminho}' (linha {erro.lineno}, "
                f"coluna {erro.colno}): {erro.msg}.")
    if isinstance(erro, csv.Error):
        return f"CSV malformado em '{caminho}': {erro}."
    if isinstance(erro, DadosInvalidosError):
        return f"Dados inválidos em '{caminho}': {erro}"
    if isinstance(erro, BackupError):
        return f"Operação cancelada, pois o backup falhou: {erro}"
    if isinstance(erro, OSError):
        return f"Erro de entrada/saída com '{caminho}': {erro.strerror or erro}."
    return f"Erro inesperado com '{caminho}': {erro}"
```

Código 8 – Função executar_seguro (erros.py)

```
def executar_seguro(operacao, caminho, *args, **kwargs):
    """Executa `operacao` sem encerrar o programa em caso de erro.


    Retorna (sucesso, resultado). Em falha, imprime a mensagem e
    resultado é None.
    """
```

```
    try:
        return True, operacao(*args, **kwargs)
    except (OSError, ValueError, csv.Error,
            DadosInvalidosError, BackupError) as erro:
        # UnicodeDecodeError e JSONDecodeError são subclasses de ValueError
        print(f"[ERRO] {descrever_erro(erro, caminho)}")
        return False, None
```

4.4 Backup automático

A função criar_backup copia o arquivo original para a pasta backups, com data e hora no nome, usando leitura e escrita binárias dentro de with open, o que preserva o conteúdo byte a byte. Ela é chamada em _preparar_destino, que é executada no início de toda função de escrita, ou seja, antes de qualquer alteração. Se a cópia falhar, uma BackupError é lançada e a escrita é cancelada, evitando a perda dos dados anteriores. Quando o arquivo de destino ainda não existe, não há o que preservar e o usuário é informado disso. Em ambos os casos o programa exibe uma mensagem.

Código 9 – Função criar_backup (backup.py)

```
def criar backup(caminho, pasta backup="backups"):
    """Copia o arquivo original para `pasta_backup` com carimbo de data/hora.

    Retorna o caminho do backup, ou None se o arquivo ainda não existe
    (nada a preservar). Lança BackupError se a cópia falhar.
    """
    if not os.path.exists(caminho):
        print(f"[BACKUP] '{caminho}' ainda não existe; backup não necessário.")
        return None

    nome, extensao = os.path.splitext(os.path.basename(caminho))
    carimbo = datetime.now().strftime("%Y%m%d_%H%M%S_%f")
    destino = os.path.join(pasta_backup, f"{nome}_{carimbo}{extensao}")

    try:
        os.makedirs(pasta_backup, exist_ok=True)
        # Cópia binária com with open: preserva o conteúdo byte a byte.
        with open(caminho, "rb") as origem, open(destino, "wb") as copia:
            copia.write(origem.read())
    except OSError as erro:
        raise BackupError(f"não foi possível copiar '{caminho}' "
                          f"({erro.strerror or erro}).") from erro

    print(f"[BACKUP] Cópia criada em '{destino}'.")
    return destino
```

Código 10 – Função _preparar_destino (escritor.py): backup antes da escrita

```
def _preparar_destino(caminho):
    """Faz o backup do arquivo existente e garante que a pasta exista."""
    criar_backup(caminho)  # ANTES de qualquer alteração
    pasta = os.path.dirname(caminho)
    if pasta:
        os.makedirs(pasta, exist_ok=True)
```

5 Testes e evidências de execução

Os testes foram executados no terminal do Visual Studio Code, a partir da pasta do projeto, com os arquivos de exemplo da pasta dados. Cada cenário abaixo descreve o procedimento, apresenta a captura de tela e comenta o resultado observado.

5.1 Leitura de CSV e conversão para JSON

Procedimento: no menu, opção 1, mantendo os caminhos padrão (origem dados/clientes.csv, destino saida/clientes.json).

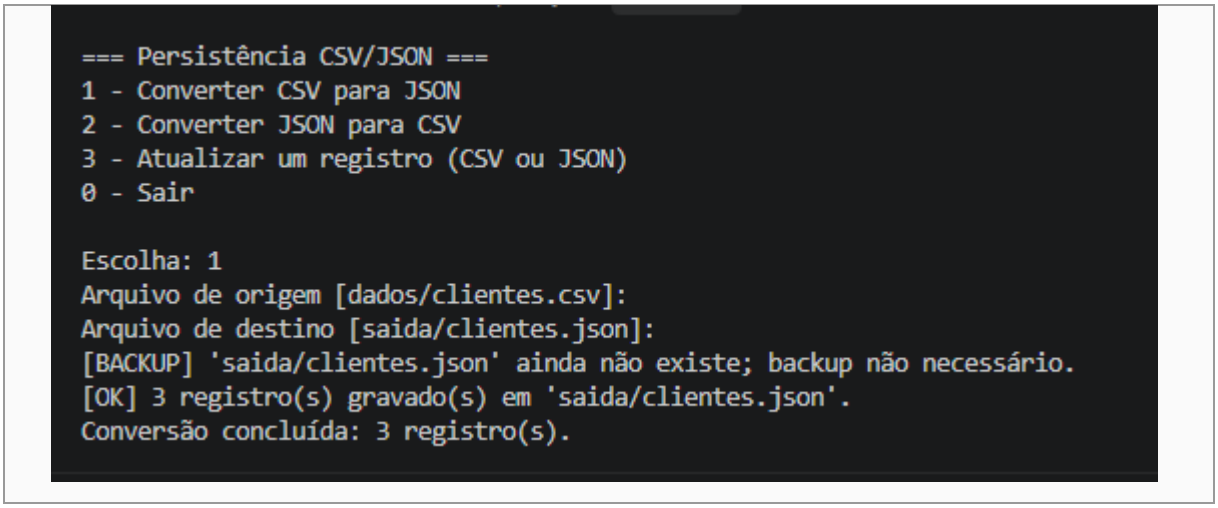


Figura 1 – Conversão do arquivo clientes.csv para JSON (opção 1 do menu).
Fonte: elaborada pelos autores (2026).

Resultado: o programa informa que o arquivo de destino ainda não existe (backup dispensado), grava três registros em saida/clientes.json e conclui a conversão. Os nomes com acento, como João e Conceição, permanecem corretos no JSON.

5.2 Leitura de JSON e conversão para CSV

Procedimento: opção 2 do menu, mantendo os caminhos padrão (origem dados/clientes.json, destino saida/clientes_convertido.csv).

```
=== Persistência CSV/JSON ===
1 - Converter CSV para JSON
2 - Converter JSON para CSV
3 - Atualizar um registro (CSV ou JSON)
0 - Sair

Escolha: 2
Arquivo de origem [dados/clientes.json]:
Arquivo de destino [saida/clientes_convertido.csv]:
[BACKUP] 'saida/clientes_convertido.csv' ainda não existe; backup não necessário.
[OK] 2 registro(s) gravado(s) em 'saida/clientes_convertido.csv'.
Conversão concluída: 2 registro(s).
```

Figura 2 – Conversão do arquivo clientes.json para CSV (opção 2 do menu).
Fonte: elaborada pelos autores (2026).

Resultado: dois registros são gravados no CSV. Como o segundo registro possui o campo email e o primeiro possui o campo tags, o cabeçalho reúne todas as colunas e os campos ausentes ficam vazios; a lista de tags é gravada como texto JSON na célula.

5.3 Atualização com geração prévia de backup

Procedimento: opção 3 do menu, com arquivo dados/clientes.csv, campo-chave id, valor 2, campo a alterar cidade e novo valor Balsas.

```
=== Persistência CSV/JSON ===
1 - Converter CSV para JSON
2 - Converter JSON para CSV
3 - Atualizar um registro (CSV ou JSON)
0 - Sair

Escolha: 3
Arquivo (.csv ou .json): dados/clientes.json
Campo-chave (ex.: id): id
Valor do campo-chave: 2
Campo a alterar: cidade
Novo valor: Fortaleza
[BACKUP] Cópia criada em 'backups\clientes_20261031_133515_720284.json'.
[OK] 2 registro(s) gravado(s) em 'dados/clientes.json'.
1 registro(s) atualizado(s).
```

Figura 3 – Atualização de um registro com criação prévia do backup (opção 3).
Fonte: elaborada pelos autores (2026).

Resultado: a mensagem de backup aparece antes da mensagem de gravação, indicando a cópia criada na pasta backups, e o programa informa que um registro foi atualizado.



Figura 4 – Pasta backups com a cópia do arquivo original e o arquivo alterado.
Fonte: elaborada pelos autores (2026).

Resultado: a pasta backups contém a cópia do clientes.csv com a cidade original, o que comprova que os dados anteriores foram preservados antes da alteração.

5.4 Tratamento de situações de erro

Procedimento: opção 1 do menu, informando naoexiste.csv como arquivo de origem.

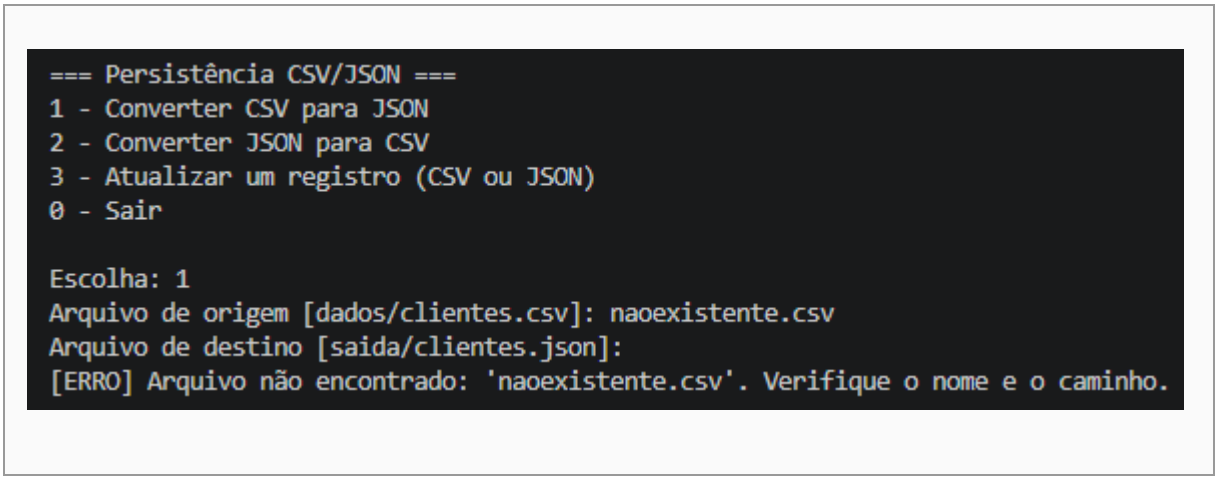


Figura 5 – Mensagem de erro para arquivo inexistente (FileNotFoundError).
Fonte: elaborada pelos autores (2026).

Resultado: o programa exibe a mensagem informando que o arquivo não foi encontrado e retorna ao menu, sem encerrar.

Procedimento adicional: opção 3 do menu, informando um arquivo com extensão diferente de .csv e .json.

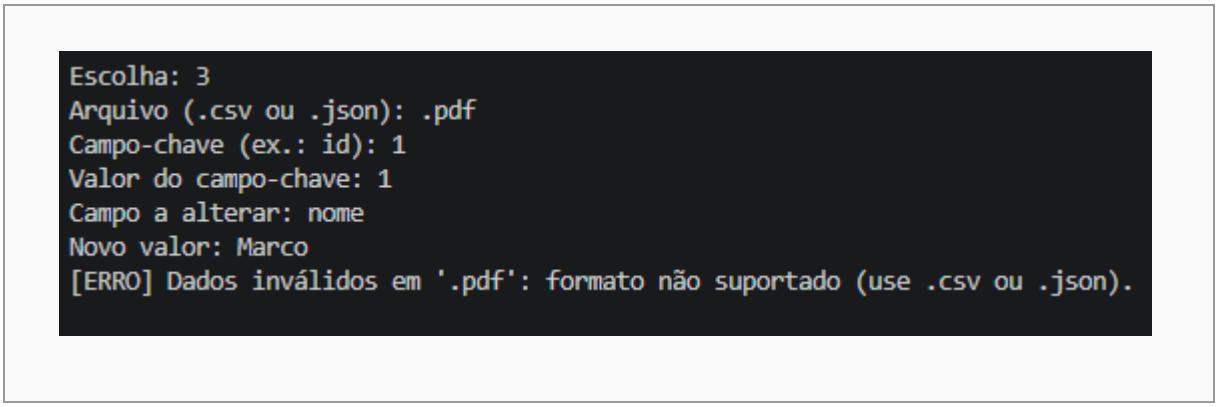


Figura 6 – Mensagem de erro para formato de arquivo não suportado.
Fonte: elaborada pelos autores (2026).

Resultado: a aplicação recusa a operação com uma mensagem clara e continua em execução.

5.5 Testes automáticos

O script testes.py executa, em uma pasta temporária, verificações de conversão nos dois sentidos, de atualização com backup (inclusive a conferência do conteúdo do backup) e de erros: arquivo inexistente, JSON malformado, arquivo fora de UTF-8, pasta no lugar de arquivo e falta de permissão. O teste de permissão é pulado em ambientes em que não é possível bloquear a leitura, como o Windows.

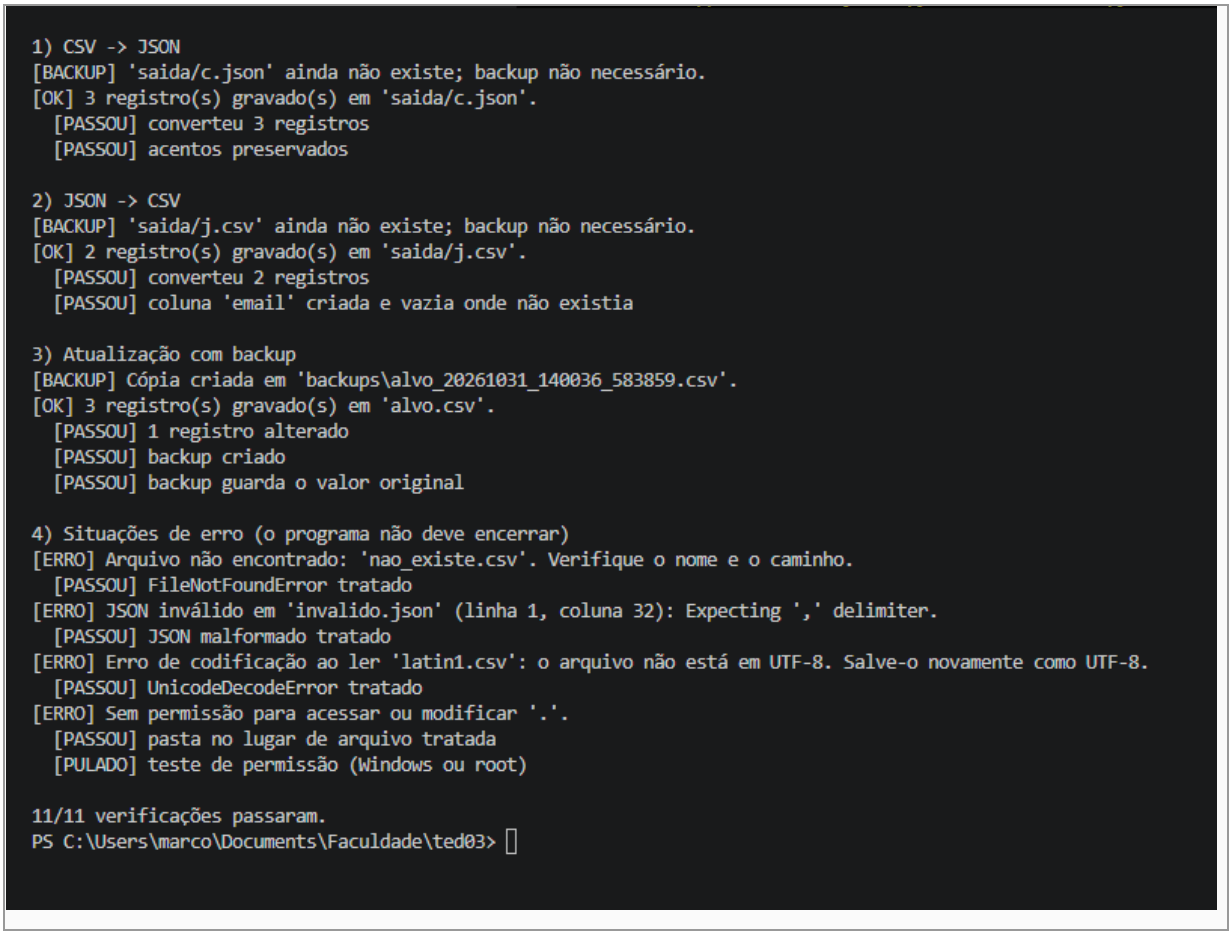


Figura 7 – Execução do script testes.py com o resultado das verificações.
Fonte: elaborada pelos autores (2026).

6 Conclusão

O módulo desenvolvido atende aos objetivos propostos: lê e converte dados entre CSV e JSON preservando registros e valores, utiliza with open com UTF-8 em todas as operações com arquivos, trata as principais exceções de E/S com mensagens claras e realiza backup automático antes de qualquer escrita ou atualização. A separação em módulos facilitou os testes e a manutenção do código.

Entre as decisões de projeto que exigiram atenção, destacam-se: a ausência de tipos no CSV, resolvida mantendo os valores como texto; a representação de listas e objetos aninhados do JSON em uma célula de CSV; a ordem correta de verificação das exceções, já que algumas são subclasses de ValueError; e a garantia de que o backup ocorra antes da gravação, cancelando-a caso a cópia falhe.

Durante o desenvolvimento, algumas dificuldades merecem registro. A primeira foi a organização do projeto em módulos: ao executar os testes, ocorreu o erro ModuleNotFoundError, causado pela estrutura de pastas incorreta, já que o pacote persistencia precisa estar no mesmo nível do script executado e conter o arquivo init.py. A segunda foi o uso de caminhos relativos no menu principal: a aplicação retornou o erro de arquivo não encontrado quando a pasta dados ainda não existia no projeto e, de modo geral, só localiza os arquivos de exemplo quando executada a partir da raiz do projeto. Esse episódio serviu, inclusive, como evidência prática do funcionamento do tratamento de exceções.

Outra dificuldade foi a diferença entre os formatos: o CSV não armazena tipos de dados, o que levou à decisão de manter os valores como texto para evitar perdas, e o JSON admite listas e objetos aninhados, que precisaram ser gravados como texto JSON dentro de uma célula do CSV. Também exigiu atenção a ordem de verificação das exceções, pois UnicodeDecodeError e JSONDecodeError são subclasses de ValueError e precisam ser tratadas antes de erros mais genéricos. Por fim, foi necessário garantir que o backup fosse realizado antes de qualquer gravação, cancelando a escrita caso a cópia falhasse, e adaptar o teste de permissão, que não pode ser reproduzido da mesma forma no Windows.

7 Referências

PYTHON SOFTWARE FOUNDATION. csv — CSV File Reading and Writing. Python 3 Documentation. Disponível em: https://docs.python.org/3/library/csv.html. Acesso em: out. 2026.

PYTHON SOFTWARE FOUNDATION. json — JSON encoder and decoder. Python 3 Documentation. Disponível em: https://docs.python.org/3/library/json.html. Acesso em: out. 2026.

PYTHON SOFTWARE FOUNDATION. Built-in Functions: open(). Python 3 Documentation. Disponível em: https://docs.python.org/3/library/functions.html#open. Acesso em: out. 2026.

PYTHON SOFTWARE FOUNDATION. Built-in Exceptions. Python 3 Documentation. Disponível em: https://docs.python.org/3/library/exceptions.html. Acesso em: out. 2026.

PYTHON SOFTWARE FOUNDATION. Input and Output: Reading and Writing Files. Python 3 Tutorial. Disponível em: https://docs.python.org/3/tutorial/inputoutput.html. Acesso em: out. 2026.